

Maker Portfolio

Synthetic Word Generator

Oscar Stomberg

Self-directed Project

August 2020



Inspiration

Summer project

The summer after quarantine started, I was on my own with not much to do. So, I invented a project for myself.

Python beginnings

Two years earlier, I started learning Python in order to make games. Now I wanted to expand on those skills.

Previous programs

When looking over old Python scripts, I discovered the first iteration of a synthetic word generator that inspired this project.

Original Synthetic Word Generator

Process

This program alternated random vowels and consonants to create words of random lengths.

Improvement

I modified my old program to be cleaner, more concise, less error-prone, and output into a text file.

Results

Though most of these are vaguely pronounceable, they do not look English: e.g. q without u.

Output:

Jekuzedecobu

Kuqobeq

Ikidiqabonakenedo

Vipale

Efoliyihesakezane

Imiwifapekod

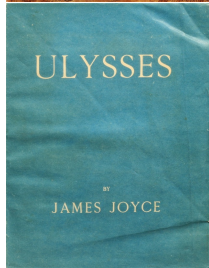
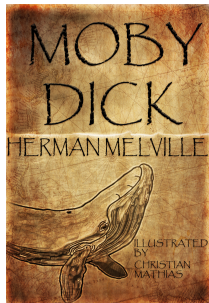
Yefudutize

Ugafubirojijicogu

Jodewidu

Deha

Using Classic Literature



The next step

To improve my synthetic word generator further would require data on what makes words sound natural.

Project Gutenberg

I downloaded classic novels such as *Moby-Dick* and *Ulysses* as plain text files through Project Gutenberg, an online repository of free eBooks.

Parsing

Using Python, I wrote a script to convert each book into a list of words, excluding formatting and punctuation.

Word Frequency Comparison

Word frequency

I improved the parsing script to instead return a list of unique words and each word's count.

Frequency comparisons

I wrote a program to combine the word lists for two books and sort them by relative frequency.

Takeaways

These tables shows how different themes, topics, and writing styles result in divergent word choice.

Table: More common in *Moby-Dick*

Word	MD/U	U/MD	Count
Whale	506.1	0.002	1246
Whales	334.0	0.003	272
Sperm	301.9	0.003	246
Mast	162.7	0.006	133
Jonah	106.0	0.009	87

Table: More common in *Ulysses*

Word	MD/U	U/MD	Count
Miss	0.009	109.1	136
Joe	0.009	110.7	138
Irish	0.011	94.6	118
Ireland	0.014	72.7	91
Bob	0.019	53.3	67

Weighted Letter Probabilities

Output:

Temasecos

Tita

Degiwepi

Qeresute

Rotomo

Onepulahad

Dite

Terosen

Ces

Tinafis

Book data

This iteration of the synthetic word generator is influenced by letter frequencies in *Moby-Dick*.

Process

Vowels and consonants alternate; each letter's probability is weighted by its frequency in natural text.

Results

Though these don't quite sound like English, using actual letter frequency makes for many more pronounceable words.

Letter Pair Frequencies

Output:

Pounesuron

Eswall

Ramasha

Darstle

Fobla

Izav

Wanu

Buspli

Rtliipi

Blyintrab

Combination analysis

To get better results, the program needed information on what letters usually follow other letters.

Data collection

Using NumPy, I wrote a program to create an array capturing the probability of each possible letter occurring after any given letter.

Results

Weighting synthetic words with these probabilities resulted in more natural sounding but sometimes less pronounceable words.

Letter Combination Analysis

Output:

Riddilect

Harly

Cro

Spinshed

Blyne

Hobbalph

Adompti

Culatch

Squed

Bolessedgerl

Letter triplets

This final iteration of the synthetic word generator uses letter triplet frequencies to pick letters based on the preceding pair.

Starting pairs

Because certain letter combinations cannot appear at the start of words, starting letter pair frequencies are used to begin words.

Results

These are almost all pronounceable and somewhat natural sounding, with real words resulting 1 in 40 times. Many sound real.

Multi-threaded Processing

Optimization

I wanted to improve the slow runtime of the programs I had written.

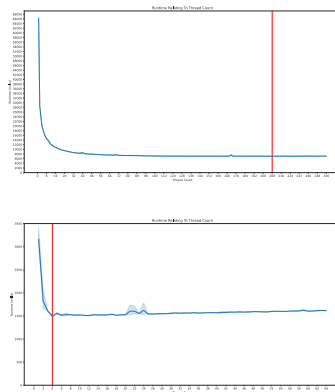
Multiprocessing

I learned to launch multiple threads of my program simultaneously, vastly improving its performance.

Matplotlib

I tested the optimal thread count and graphed the results.

Figure: Graphs of optimal threading; runtime versus thread count



What I Learned

Python programming

Throughout this process I greatly improved my skill at writing clean, concise code. I also learned how to use new libraries such as NumPy and Matplotlib.

Data analysis

I also learned a lot about using computers to create and handle large amounts of data and how to structure such data, as well as the basics of probability and statistics.

Computational linguistics

At its core, this project was an exploration, through my Python skills, of what makes English words sound English.

Color Palettes From Images

Taking it further

The Python data processing skills I learned have been used in many other projects since.

ImagePy

I started this project to create pleasing color schemes for my Linux terminal based on my desktop background image.

Techniques learned

This taught me a lot about image processing and color theory.

Figure: Color palette generated from my desktop background

